**BBM418-AIN433 : Computer Vision Laboratory**
Fall 2025
Instructor: Assoc. Prof. Dr. Ali Seydi KEÇELİ
TA's: S. Meryem Taşyürek* & M. Aslı Taşgetiren

# Assignement 4: Implementing a Lightweight Object Detector from a Research Paper

Due date: Thursday, 25-12-2025, 11:59 PM.

The objective of this assignment is to implement the core architecture of the **DETR** (DEtection TRansformer) model as described in the original research paper. **You will carefully read the paper, analyze its main components, and reconstruct the model following the ideas and mechanisms introduced by the authors.**

To ensure that the model can be trained within the constraints of this course, you will implement a lightweight configuration of DETR—using fewer encoder/decoder layers and a compact backbone—while preserving the paper's conceptual framework: transformer-based object detection, object queries, set-based predictions, and Hungarian matching. The goal is not to modify the method itself, but to adapt the architecture to a smaller, trainable version that still reflects the principles presented in the paper.

Your **Tiny-DETR model** will be trained and evaluated on the **PennFudanPed** dataset. You will experiment with different lightweight backbones and hyperparameters, analyze their influence on performance, and document your observations. Throughout the assignment, the emphasis is on understanding the paper, reconstructing its model, and explaining how your design choices relate to the original work.

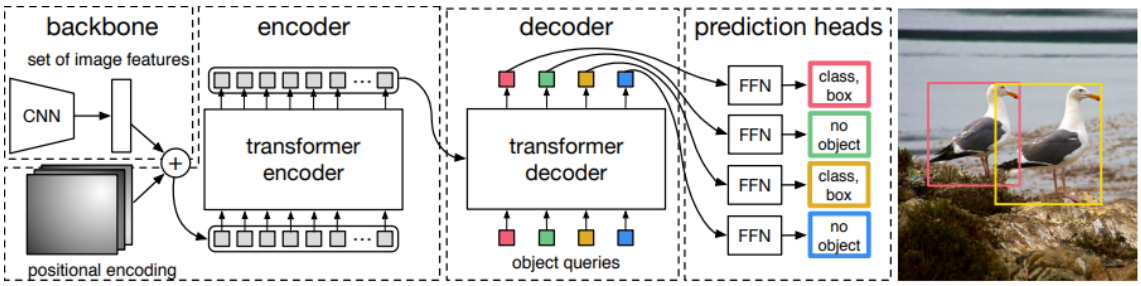


Figure 1: DETR employs a CNN backbone to extract a 2D feature map, flattens it, and adds positional encodings before feeding it into a transformer encoder. A transformer decoder then processes a fixed set of learned object queries while attending to the encoder output. Each decoder output is passed through a shared feed-forward network to predict a class and bounding box, or a "no-object" label. Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). *End-to-End Object Detection with Transformers*. arXiv:2005.12872.

# 1 Dataset & Setup (15 Points)

This assignment uses the **PennFudanPed** dataset, a small pedestrian detection dataset commonly used for lightweight object detection experiments.

- **Dataset:** `https://www.cis.upenn.edu/~jshi/ped_html/PennFudanPed.zip`
- 170 RGB images
- Bounding-box annotations stored in corresponding `.txt` files
- Single object class: *pedestrian*

Figure 2: Example images from the PennFudanPed dataset. Each image contains one or more pedestrian instances with bounding-box annotations.

After downloading, organize the dataset as follows:

```
PennFudanPed/
    Pedestrian/
        FudanPed00001.png
        FudanPed00001.txt
        ...
```

You are required to parse the annotation files and convert them into the prediction target format of your Tiny-DETR model (normalized bounding box coordinates and class label).

**Fixed Dataset Split.** Because PennFudanPed is a very small dataset, we provide a fixed split that keeps the training set intentionally large to allow stable optimization. You must use the official split provided in `train.txt`, `val.txt`, and `test.txt`, corresponding to approximately 70% training, 15% validation, and 15% test data. These provided text files are containing a list of image file names (one per line). You must use these splits exactly as given when constructing your dataset and dataloaders.

All models must be trained using a **fixed input resolution** of 512x512. Basic data augmentations such as horizontal flip, random resize, or slight color jitter are recommended to improve generalization. The specific augmentation strategy you choose must be documented and justified in your report.

The compact size of PennFudanPed enables **multiple training experiments within a few hours**, allowing you to compare backbone architectures, transformer configurations, and hyperparameter selections.

# 2 Paper-Guided Analysis & Tiny-DETR Design (30 pts)

In this assignment, you are expected to read and interpret the original DETR paper and use its architectural principles to design and implement a lightweight variant suitable for training on the PennFudanPed dataset. Your work should demonstrate that you understand how DETR formulates object detection as a set prediction problem, how its major components interact, and how these ideas guide the structure of your compact model. The goal is to show a clear connection between the reasoning presented in the paper and the design choices you make in your Tiny-DETR implementation.
Your report must address the following high-level components, each grounded in insights drawn from the paper:

1. **Backbone Analysis and Comparison**
   DETR relies on convolutional feature extractors to produce spatial feature maps suitable for transformer processing. In this assignment, you must use the following lightweight backbones:

   - **MobileNetV2 (ImageNet-pretrained)**
   - **ResNet18 (ImageNet-pretrained)**

   You are required to train Tiny-DETR with both backbones and compare their performance. Your analysis should explain, using insights from the paper, how backbone capacity, feature dimensionality, and representational strength influence transformer behavior.

2. **Positional Encoding**
   Explain why positional information is essential for transformers in object detection. Summarize the sinusoidal positional encoding described in the paper and implement an appropriate encoding module for your lightweight design.

3. **Transformer Encoder–Decoder Structure**
   Analyze the purpose of multi-head attention, encoder layers, and decoder layers within DETR. Based on the computational constraints of the assignment, design a reduced-depth transformer (typically 2–3 encoder layers and 2–3 decoder layers) that still reflects the logic of the original method.

4. **Object Queries and Set-Based Predictions**
   Describe how object queries function as learned embeddings and how DETR produces a fixed-size set of predictions. You must experiment with multiple query counts and analyze their effect on detection performance.

5. **Hungarian Matching and Loss Formulation**
   Discuss how DETR uses bipartite matching to enforce one-to-one assignments between predictions and ground truth. Implement a matching-based loss and justify the loss weights you choose for classification and bounding-box regression.

6. **Training Under a Small Dataset Regime**
   Reflect on the challenges of training transformer-based detectors on small datasets. Explain how design decisions such as reduced model capacity, pretrained backbone weights, and data augmentation influence model convergence. You must apply at least two augmentation strategies and report their effect.

7. **Summary of Experimental Findings**
   Provide a comparative summary showing how different backbones, query counts, and augmentation strategies affect model performance. Your evaluation must include mAP@0.5 scores and representative qualitative results.

Throughout your implementation, you may not use the official DETR codebase which you will find is not helpful at all in this context or any pretrained detection framework. All architectural decisions must be justified using observations derived from the DETR paper and supported by the experiments you conduct.

# 3 Training Procedure & Evaluation (40 pts)

This section specifies how you are expected to train and evaluate your Tiny-DETR model. The goal is to keep the training process comparable across students while still leaving room for informed design choices. You should follow the steps below and clearly document all decisions in your report.

## Training Procedure (Step-by-Step)

**Step 1: Data Split and Preprocessing.** Create a fixed train/validation/test split for the PennFudanPed dataset and use the same split for all experiments. Resize all images to a single resolution ($512 \times 512$) and normalize them consistently. Briefly describe your split strategy and preprocessing choices.

**Step 2: Baseline Configuration.** Define an initial Tiny-DETR configuration using one of the specified backbones (MobileNetV2 or ResNet18), a reasonable transformer depth, a hidden dimension, and an initial number of object queries. You should choose hyperparameters (learning rate, batch size, number of epochs) within realistic ranges (e.g., tens of epochs, small batch sizes) so that training can be completed in a few hours. This configuration will serve as your *baseline model*.

**Step 3: Baseline Training.** Train your baseline Tiny-DETR model on the training set and monitor performance on the validation set. Record training and validation curves (e.g., loss, mAP@0.5) and note any signs of overfitting, underfitting, or instability. Your report should summarize what you observe from this initial training run.

**Step 4: Guided Improvements.** Starting from your baseline, you must perform at least two targeted modifications to improve performance. The following types of changes are expected:

- switching between the two backbones (MobileNetV2 vs. ResNet18),
- adjusting the number of object queries,
- introducing or refining data augmentation,
- making a controlled change in transformer capacity (e.g., depth or hidden size),
- tuning the learning rate or loss weight configuration.

Each change should be motivated using insights from the DETR paper and justified as an attempt to improve training behavior on a small dataset.

**Step 5: Final Model Selection and Reporting.** Based on your experiments, select one configuration as your final Tiny-DETR model. You should explain why this configuration is preferable to your baseline and how each modification influenced performance. Your discussion must connect design choices, training dynamics, and evaluation results.

## Training Duration

Because Tiny-DETR is designed to be lightweight and the PennFudanPed dataset is very small, you are expected to train your models for approximately **20–30 epochs**. Training beyond 30 epochs typically leads to overfitting without meaningful gains, while fewer than 15 epochs is usually insufficient for stable convergence. However, if you believe that your model requires *more* or *fewer* epochs—based on validation curves, early overfitting, slow convergence, or other experimental observations—you may adjust the number of epochs accordingly. In such cases, you must clearly explain and justify your reasoning in the report.

## Evaluation Criteria

All models will be evaluated using the **mAP@0.5** metric on the test split. To satisfy the performance requirement of this assignment, your final Tiny-DETR model is expected to achieve at least

$$\textbf{mAP@0.5} \geq 0.50.$$

**If your model does not reach this threshold**, you must provide a clear analysis of the failure modes, supported by experimental evidence (e.g., learning curves, qualitative examples, comparison to the baseline). A well-argued explanation of why certain design choices did not work will be taken into account in grading.

Note that the original DETR results are reported on the COCO dataset with large-scale training. Therefore, you should not expect to match the paper's AP values in this assignment. Our target is a reasonable mAP@0.5 on a much smaller single-class dataset.

On this small dataset, a correct Tiny-DETR implementation that follows the instructions will typically reach around **mAP@0.5 = 0.50**. If your apply stronger data augmentation, a stable learning rate schedule, and a well tuned number of object queries, you can exceed **0.60** usually. With careful design choices and consistent training, it is possible to reach even higher scores. You are encouraged to aim for **0.60+**, as this generally reflects not only a correct implementation but also a thoughtful application of the principles described in the DETR paper.

Beyond meeting the minimum threshold, you are expected to show that you actively tried to improve your model. Your report should:

- compare MobileNetV2 and ResNet18 backbones under otherwise similar settings,
- analyze the effect of at least one change in query count or transformer capacity,
- demonstrate the impact of your augmentation strategy on validation or test mAP,

- include both quantitative results (mAP@0.5, training time per epoch) and qualitative detection outputs (example images with predicted boxes).

The emphasis is not only on reaching a certain accuracy, but on demonstrating a systematic training process: starting from a reasonable baseline, applying informed modifications, and critically evaluating how each decision affects your Tiny-DETR implementation on PennFudanPed.

**Final Performance & Evaluation (10 Points)**

Your final Tiny-DETR model must reach:

$$\textbf{mAP@0.5} \geq 0.50$$

- **Meets or exceeds threshold (8 pts)** Reliable mAP@0.5 performance on the test split.
- **Qualitative Detections (2 pts)** Clean, clear qualitative examples inside the report (good + failure cases).

*Note:* If a your model does *not* reach the threshold but you provide an excellent failure-mode analysis, partial credit (up to 5 pts) may still be awarded.

# 4 Report and Analysis (15 Points)

You must submit a single, clearly written `report.pdf` that explains: *what you implemented*, *why you designed your Tiny-DETR model the way you did*, and *how you evaluated its performance*. Your grade depends on clarity, reasoning, and the quality of visual/quantitative evidence.

**Required Structure**

1. **Overview (1–2 paragraphs).** Briefly restate the goal of the assignment and summarize your full Tiny-DETR pipeline (backbone, transformer, object queries, loss, training flow). Highlight what aspects of the DETR paper were most important for your design.
2. **Dataset & Setup.** Describe:
   - the PennFudanPed dataset,
   - your train/validation/test split,
   - image resolution and preprocessing choices,
   - data augmentation strategy,
   - folder layout of your implementation.
3. **Model Architecture.** Explain your Tiny-DETR design in relation to the DETR paper **as stated in Section 2**.
4. **Training Procedure (Refer to the Section 3)** Summarize:

- your baseline configuration (epochs, LR, optimizer, LR schedule),
- training curves and stability observations,
- any issues such as overfitting, slow convergence, or imbalance.

Clearly identify your *baseline model* and your *final improved model*.

5. **Experiments & Ablation (Refer to the Section 3).** Your report must include the following experiments:

   - **Backbone Comparison:** MobileNetV2 vs. ResNet18 (same training settings).
   - **Query Count Ablation:** at least two different values (e.g., 50 vs. 100).
   - **Augmentation Impact:** with vs. without augmentation.
   - **Optional:** transformer depth or hidden dimension variation.

   Each experiment should include a short explanation of: *why you tried it*, *what changed*, and *how it affected mAP or training behavior*.

6. **Results (Figures & Tables).** Include the following:

   - *Qualitative detections:* 6–8 test images with predicted boxes (good + failure cases).
   - *Training curves:* loss and validation mAP.
   - *Backbone comparison table:* MobileNetV2 vs. ResNet18 mAP, training time/epoch.
   - *Query count table:* different query sizes vs. mAP.
   - *Augmentation table:* augmentation vs. no-augmentation.
   - *Final result:* test mAP@0.5 showing your best-performing model.

7. **Discussion (Accuracy, Robustness, Limitations).** Analyze:

   - when your Tiny-DETR performs well vs. fails (e.g., small objects, occlusion),
   - effect of dataset size,
   - transformer behavior with limited data,
   - limitations of your model and training choices.

8. **Reproducibility Notes.** Provide:

   - all key parameters (LR, batch size, epochs, loss weights),
   - random seeds,
   - hardware used (GPU type),
   - exact command to run training/evaluation.

# What to Hand In

Your submission must contain the following items in a clean and organized structure.

- `report.pdf` – your main written report following the required structure in Section 5.
- `b<studentNumber>.ipynb` (or a set of well-structured `.py` files) – your complete Tiny-DETR implementation.
- `demo.mp4` – a short presentation video (see "Demo Video Requirements" below).

All qualitative detection images, tables, training curves, and ablations must be included directly inside your `report.pdf`. *No separate image folders are required.*

**Submission Size Limitation** The course submission system accepts files up to **8 MB**. If your `report.pdf` or `demo.mp4` exceeds this limit, you must upload the large files to an external service such as Google Drive, OneDrive, or Dropbox:

- Place `report.pdf` and `demo.mp4` together inside a folder in your Drive.
- Set the folder to "anyone with the link can view."
- Include the Drive link with a placeholder PDF.

In that case, the only files uploaded to the submission portal should be:

`b<studentNumber>.ipynb` (or .py) + `a small placeholder report.pdf`

The placeholder PDF should simply contain:

```
This report exceeds the 8 MB submission limit.
The full report and demo video are available at:
<your_drive_link_here>.
```

**Submission Format** Package your files into a single archive named:

`b<studentNumber>.zip`

and upload it to:

`https://submit.cs.hacettepe.edu.tr`

**Demo Video Requirements (Mandatory, 5 Minutes Maximum)** You must prepare a brief presentation video called `demo.mp4`. It should focus on explaining:

- the overall goal of the assignment and the Tiny-DETR pipeline you built,
- your model architecture (backbone, transformer, queries, loss),
- training procedure and major observations from learning curves,
- qualitative detection examples (good and challenging cases),
- your ablation insights and final performance.

A simple screen recording with narration (e.g., Zoom, OBS, PowerPoint recorder) is sufficient. Audio must be clear, and the video must be under 5 minutes. If the video file is large, upload it to Drive and include the link inside `report.pdf`.

**Checklist Before Submission**

- All required figures and tables appear inside `report.pdf`.
- Your notebook or script runs end-to-end and reproduces reported results.
- If needed, external Drive links for `report.pdf` and/or `demo.mp4` are valid and publicly accessible.
- The final archive is named correctly as `b<studentNumber>.zip`.

No additional files are required. Your written report, implementation, and demo video together must clearly demonstrate your understanding and execution of the Tiny-DETR assignment.

# Academic Integrity

All work on assignments must be done individually unless stated otherwise. You are encouraged to discuss with your classmates about the given assignments, but these discussions should be carried out in an abstract way. That is, discussions related to a particular solution to a specific problem (either in actual code or in the pseudocode) will not be tolerated. In short, turning in someone else's work, in whole or in part, as your own will be considered as a violation of academic integrity. Please note that the former condition also holds for online/AI sources. Make use of them responsibly, refrain from generating the code you are asked to implement. Remember that we also have have access to such tools, making it easier to detect such cases.